

Phase 1 To Do List

1. Project Setup & Repository Foundation

Create a public GitHub repo named debate-market. Initialize folder structure: /contracts (Solidity escrow + settlement), /agents (Bull, Bear, Judge agent logic), /axl-nodes (3 separate AXL node configs), /keeper (KeeperHub integration), /uniswap (swap execution module), /frontend (Next.js dashboard), /tests, /utils. Set up .env.example with placeholders for Groq API key, Alchemy RPC, KeeperHub credentials, AXL node configs, and Uniswap API keys. Initialize with npm and pip dependency files. Set up Hardhat in /contracts. This is the only setup step — every subsequent step is pure building.

2. Deploy Escrow Smart Contract

Write DebateEscrow.sol in /contracts. This contract: accepts deposits from users who choose Bull or Bear side, locks funds for the duration of a debate session, tracks each side's total staked amount, exposes a settleSide(winner) function callable only by a designated settlementExecutor address (this will be KeeperHub's wallet), and emits events Deposited(user, side, amount) and Settled(winningSide, totalPayout). Deploy to Unichain Sepolia testnet using Hardhat. Capture and store contract address. Verify on the Unichain Sepolia explorer. This contract is the financial backbone — all fund movement flows through it.

3. Deploy Conviction Tracker Smart Contract

Write ConvictionTracker.sol that stores the running conviction scores for Bull and Bear agents onchain. Exposes updateConviction(bullScore, bearScore, roundNumber) callable only by the Judge agent's wallet. Emits ConvictionUpdated(round, bullScore, bearScore) events. Stores a full history of all rounds so the frontend can replay the entire debate. Also stores debateActive boolean — when conviction delta exceeds a configurable threshold (e.g., one side hits 70+), it flips debateActive = false and signals that settlement should be triggered. Deploy alongside DebateEscrow.sol on Unichain Sepolia. Store ABI in /contracts/abi/.

4. Set Up Three Gensyn AXL Nodes

Install AXL binary from the Gensyn GitHub repo on your local machine. Spin up three separate AXL node instances on different ports (e.g., 8001, 8002, 8003) — one for Bull Agent, one for Bear Agent, one for Judge Agent. Each node gets a unique peer identity. Configure them to discover each other on the local mesh. Test peer discovery by sending a test message from Node 1 → Node 3 and verifying it arrives encrypted. Document node startup commands in /axl-nodes/README.md. This step proves the AXL requirement: all three agents communicate peer-to-peer with no central broker. All subsequent agent messages must go exclusively through these AXL nodes.

5. Build the Bear Agent Core Logic

In `/agents/bear_agent.py`, set up a LangChain agent using Groq's free API (llama3-70b-8192 model — fast and free). Write a system prompt: "You are a Bear analyst. Given token market data, construct a concise bearish argument using onchain evidence. Output JSON: {argument: string, confidence: int 0-100, keyMetrics: []}." Connect to Alchemy free tier to pull live data: 24h price change, volume delta, wallet outflow counts from The Graph. Feed this data as context to the agent each round. Agent outputs structured JSON argument. Publish this JSON as a message to the Judge Agent's AXL node over HTTP to `localhost:8003`. Log all outputs to SQLite.

6. Build the Bull Agent Core Logic

Mirror of step 5 but bullish. In `/agents/bull_agent.py`, same LangChain + Groq setup. System prompt: "You are a Bull analyst. Given token market data, construct a concise bullish argument using onchain evidence. Output JSON: {argument: string, confidence: int 0-100, keyMetrics: []}." Pull the same onchain data but frame interpretation as bullish signals — whale accumulation, volume spikes, positive price momentum, upcoming protocol catalysts (from The Graph event data). Publish JSON argument to Judge's AXL node (`localhost:8003`). Critically — Bull and Bear agents never talk to each other directly, only through the Judge. This enforces clean separation and satisfies AXL's requirement of distinct node communication.

7. Build the Judge Agent Core Logic

In `/agents/judge_agent.py`, set up a third LangChain + Groq agent listening on AXL Node 3 (`localhost:8003`). This agent: receives both Bull and Bear JSON arguments each round, scores them against five criteria (evidence quality, logical consistency, metric accuracy, predictive value, argument clarity — each 0-20 points), outputs a verdict JSON: {winner: "bull"|"bear", bullScore: int, bearScore: int, reasoning: string, roundNumber: int}. Critically, the Judge agent also checks the previous round's prediction against actual price movement (did the winner's prediction come true?) and adds a 10-point accuracy bonus. This creates a feedback loop where agents that cited better evidence get rewarded over time. Publish verdict back to both Bull and Bear nodes via AXL.

8. Build the Debate Orchestrator

In `/agents/orchestrator.py`, build the master loop that runs a full debate session: (1) fetch fresh market data, (2) trigger Bull Agent (POST to AXL node 1), (3) trigger Bear Agent (POST to AXL node 2), (4) wait for both arguments (poll AXL node 3 inbox), (5) trigger Judge Agent with both arguments, (6) receive verdict from Judge, (7) update conviction scores in SQLite, (8) call `ConvictionTracker.sol updateConviction()` with new scores via ethers.js, (9) check if debate end condition is met (one side hits 70+ conviction), (10)

if yes, trigger settlement flow. Add a configurable `ROUND_INTERVAL` (default 60 seconds). Log every round with full message trace — input data, both arguments, verdict, updated scores — to SQLite for the audit trail.

9. Build the Onchain Data Feed Module

In `/utils/market_data.py`, build a data aggregator that fetches for any given token: (1) Price + 24h change via Alchemy `eth_call` to Uniswap v3 pool's slot0; (2) Wallet flows via The Graph (free) — query large transfers in last 2 hours; (3) Pool liquidity depth via Uniswap v3 subgraph — total TVL, recent LP additions/removals; (4) Funding rate proxy — calculate price deviation between spot and a reference index. Normalize all data into a single `MarketSnapshot` dataclass that both Bull and Bear agents receive identically each round. This is crucial — both agents see the same raw data but interpret it differently, making the debate fair and the Judge's job meaningful. Cache snapshots locally to avoid hitting free tier rate limits.

10. Build the Uniswap Swap Execution Module

In `/uniswap/swap_executor.py`, integrate the Uniswap API to handle two types of swaps: (1) Mid-debate micro-settlements — after each round, a small % (configurable, default 2%) of the losing side's escrow is swapped to the winning side's token via Uniswap. This happens every round. (2) Final settlement swap — when debate ends, the full redistribution is executed via Uniswap. Use the Uniswap API's `/quote` endpoint to get optimal routing before every swap. Use `/swap` endpoint to build the calldata. Execute via `ethers.js` with the agent's wallet. Log every swap: quote received, route taken, actual execution price, slippage realized. This generates continuous Uniswap API usage throughout the debate — not just a one-time call — which is exactly what they want to see.

11. Integrate KeeperHub for All Swap Execution

Replace direct `ethers.js` swap calls from Step 10 with KeeperHub MCP integration. In `/keeper/execution_handler.py`, route every swap through KeeperHub's MCP server. KeeperHub now owns: retry logic if gas spikes mid-debate round, MEV protection on final settlement (nobody can frontrun the payout), private routing for large settlement swaps, and the full audit trail of every transaction. Implement KeeperHub's job queue: each swap is submitted as a job with `maxGasPrice`, `deadline`, and `retryPolicy`. Poll job status until confirmed. Store KeeperHub job IDs alongside Uniswap swap data in SQLite. This creates a verifiable chain: debate verdict → conviction update → KeeperHub job → Uniswap swap → onchain settlement.

12. Add Agent Memory and Historical Learning

Enhance Bull and Bear agents with LangChain ConversationBufferMemory that persists across rounds within a session. Each agent remembers: its last 5 arguments, which ones the Judge scored highly, which onchain metrics correlated with winning rounds. Add a StrategyAdapter class that analyzes the agent's own win/loss history after every 3 rounds and adjusts its argument weighting — if whale wallet data has been winning, weight it higher next round. Store full memory snapshots in SQLite. This makes agents visibly smarter over a long debate session — early rounds show generic arguments, later rounds show refined, evidence-heavy arguments. This is the detail that makes judges say "this is genuinely autonomous."

13. Build Risk Management and Debate Safety Module

In `/utils/risk_manager.py`, implement debate-level safety checks: (1) Stale data guard — if market data is >2 minutes old, pause the round and wait for fresh data before agents argue; (2) Conviction drift limit — if conviction moves >15 points in a single round (suspicious), flag it and require Judge to provide extended reasoning; (3) Minimum stake threshold — debate only proceeds if total staked value on both sides exceeds a minimum (prevents 1-sided debates); (4) Debate timeout — if no conviction winner after 20 rounds, trigger a draw settlement (funds returned minus gas); (5) AXL message validation — verify every AXL message has a valid sender signature before the Judge processes it, rejecting spoofed arguments. Log every safety trigger to SQLite.

14. Wire the Complete End-to-End Pipeline

Connect all modules into the single runnable pipeline. Running `python orchestrator.py --token ETH --duration 10rounds` should: start all 3 AXL nodes, open the escrow contract for deposits (2-minute window), begin round loop (fetch data → Bull argues → Bear argues → Judge scores → conviction updates onchain → KeeperHub queues swap → Uniswap executes micro-settlement), display live conviction scores in terminal, and on debate end — trigger final KeeperHub settlement job → Uniswap final swap → escrow distributes winnings. Add `--dry-run` flag that skips real transactions but runs full logic with simulated wallet calls. Test dry-run with 5 rounds on ETH/USDC pair. Fix all integration bugs before moving to testnet.

15. Write Integration Tests with Synthetic Market Data

In `/tests/integration_test.py`, build a test suite that replays synthetic market scenarios: (1) Bull dominant market — price up 15%, whale accumulation; verify Bull wins majority of rounds; (2) Bear dominant market — price down 12%, large outflows; verify Bear wins; (3) Choppy market — alternating signals; verify conviction stays near 50, no early trigger; (4) AXL failure simulation — kill one AXL node mid-debate; verify orchestrator detects and pauses; (5) KeeperHub gas spike — mock KeeperHub returning gas-too-high; verify retry logic activates; (6) Judge consistency — feed identical arguments

twice; verify same score both times (determinism check). Target >90% test pass rate. Run tests in CI via GitHub Actions on every push.

16. Deploy and Run on Unichain Sepolia Testnet

Deploy DebateEscrow.sol and ConvictionTracker.sol to Unichain Sepolia (free testnet ETH from faucet). Seed the escrow with 0.1 test ETH on each side. Configure all agents to use Unichain Sepolia RPC via Alchemy free tier. Run a full live debate session — 10 rounds, real AXL messages, real onchain conviction updates, real Uniswap testnet swaps routed through KeeperHub. Capture every transaction hash. Verify conviction update txs on Unichain Sepolia explorer. Verify Uniswap swap txs. Verify final KeeperHub settlement tx. Screenshot all explorer confirmations for submission. This generates your proof-of-work: a chain of 15+ real transactions showing the full debate → conviction → swap → settlement cycle.

17. Build the Live Frontend Dashboard

In /frontend build a Next.js app (free) with: (1) Live Debate View — two panels side by side showing Bull and Bear's latest argument text, updating each round; (2) Conviction Meter — animated bar showing Bull vs Bear conviction scores, pulling from ConvictionTracker.sol events via Alchemy websocket; (3) Stake Panel — shows total staked on each side, user can connect wallet and deposit; (4) Round History — scrollable feed of past rounds with Judge's verdict and score for each; (5) Transaction Log — live feed of KeeperHub job IDs and Uniswap swap hashes with explorer links. Deploy frontend on Vercel free tier. This makes the demo video dramatically more impressive — judges see a real UI, not just terminal logs.

18. Stress Test Agent Argument Quality

Run 50 synthetic debate rounds and manually audit argument quality. Common failure modes to fix: (1) agents citing the same metric as both bullish and bearish (fix by adding metric-to-sentiment mapping in system prompt); (2) Judge giving identical scores every round regardless of argument quality (fix by adding explicit scoring rubric in Judge prompt with examples); (3) agents producing invalid JSON (fix with LangChain output parsers + retry on parse failure); (4) conviction never moving beyond 55/45 (fix by increasing Judge's scoring sensitivity). After fixes, run 20 more rounds and verify conviction reaches 70+ within 15 rounds on a clearly directional synthetic dataset. This step ensures the demo video shows a decisive, interesting debate — not a flat line.

19. Security Hardening and Final Validation

Before submission: (1) run Slither on both Solidity contracts — fix any high/medium severity findings; (2) confirm no private keys or API keys are in git history (git log --all search); (3) verify settleSide() in DebateEscrow.sol is truly only callable by KeeperHub's address — test calling it from another wallet and

confirm revert; (4) verify AXL message signing — send a spoofed message to Judge's node and confirm it gets rejected; (5) run full pipeline one final time on testnet from scratch (fresh wallet, fresh contract deploy) and confirm everything works without any manual intervention. Record this clean run — it becomes the basis for your demo video. Confirm all 3 tracks are clearly exercised in this run: AXL messages visible, Uniswap swaps confirmed, KeeperHub jobs completed.

20. Documentation, Demo Video & Submission

Write the README.md with: system architecture Mermaid diagram showing Bull→AXL→Judge←AXL←Bear→KeeperHub→Uniswap flow, local setup in 3 commands, deployed contract addresses, all transaction hashes from testnet run, and how each partner track is used. Create FEEDBACK.md (mandatory for Uniswap prize) documenting your honest experience with the Uniswap API. Record 2–4 minute demo video: 20s problem statement → 60s live debate round in the frontend UI → 40s KeeperHub job executing → 40s Uniswap swap confirmed on explorer → 20s conviction final settlement. No AI voiceover, minimum 720p. Submit via Hacker Dashboard by May 3rd 2026 12:00pm EDT selecting Uniswap + Gensyn + KeeperHub tracks. Also submit a KeeperHub feedback write-up for the \$500 Builder Feedback Bounty.